

IDFC FIRST Bank Edge Delivery Services - Admin Guide v3

Date: 2026-04-23

Adobe

Table of Contents

IDFC FIRST Bank – Edge Delivery Services Admin Guide	3
1. Introduction	3
2. Quick Reference	3
3. Authentication and Access	4
4. User Management	5
5. Content Operations	7
6. Path Mappings	9
7. Cache Management	10
8. Code Operations	11
9. Config Service Management	12
10. AEM Author Administration	13
11. Indexing and Sitemap	15
12. Monitoring and Troubleshooting	16
13. Security Configuration	18
14. Resources	20

IDFC FIRST Bank – Edge Delivery Services Admin Guide

1. Introduction

This guide is the definitive administrative reference for the IDFC FIRST Bank Edge Delivery Services project. The project uses the **AEM Edge Delivery Services crosswalk (xwalk) architecture**, where content is authored in the **AEM Author instance** using the **Universal Editor** and delivered through the Edge Delivery CDN.

Dual Administration Surface

Administrators of this project operate across two distinct control planes:

- **AEM Author Instance** – Manages content authors, page trees, DAM assets, workflows, and permissions within the AEM JCR repository. This is where all content creation and editing occurs via the Universal Editor.
- **Edge Delivery Config Service** – Manages the Edge Delivery CDN layer, including preview and publish operations, cache management, code sync, site configuration, and admin/author user access for the delivery tier.

Understanding both surfaces is essential. A content change made in AEM Author does not appear on the Edge Delivery CDN until an explicit preview or publish operation is triggered. Similarly, code changes pushed to GitHub do not take effect until a code sync is executed.

Content Flow

The content lifecycle in this xwalk project follows this path:

1. Content is created or edited in **AEM Author** via the **Universal Editor**
2. An admin or author triggers a **preview** operation via the Sidekick or Admin API
3. Content appears on the preview tier (aem.page)
4. An admin or author triggers a **publish** operation
5. Content is live on the production tier (aem.live)

2. Quick Reference

Key URLs

Resource	URL
AEM Author	https://author-idfc-stage-65.adobecqms.net
Preview	https://main-idfc-aemsites.aem.page/
Live	https://main-idfc-aemsites.aem.live/
Admin API Base	https://admin.hlx.page/
Config Service	https://admin.hlx.page/config/aemsites/sites/idfc.json

Project Identifiers

Property	Value
Organization	aemsites
Site	idfc
Branch	main
GitHub Repository	aemsites/idfc
Content Source Type	markup (AEM JCR)
Content Root	/content/idfc-edge/
DAM Root	/content/dam/idfc-edge/
Production Domain	https://www.idfcfirst.bank.in

Content Source Configuration

In this xwalk project, the content source is **not** configured through a file in the repository. Instead, the content source – the AEM Author markup endpoint – is configured through the **Edge Delivery Config Service**. This is a key difference from document-based Edge Delivery projects.

The Config Service stores the content source configuration as part of the site-level configuration at `https://admin.hlx.page/config/aemsites/sites/idfc.json`. The relevant section is:

```
{
  "contentSource": {
    "url": "https://author-idfc-stage-65.adobecqms.net/bin/franklin.delivery/aemsites/idfc/main",
    "type": "markup",
    "suffix": ".html"
  }
}
```

This configuration tells Edge Delivery to fetch content from the AEM Author instance's Franklin delivery servlet rather than from a document-based source. To view or update this configuration, use the Config Service API (see Section 9).

3. Authentication and Access

AEM Author Login

Access to the AEM Author instance at `https://author-idfc-stage-65.adobecqms.net` is managed through Adobe IMS (Identity Management System). Administrators and authors authenticate using their Adobe ID credentials. Access is granted through:

- **Adobe Admin Console** – Organization-level user and product profile management
- **AEM User/Group Administration** – Instance-level permissions and group memberships
- **IMS Integration** – Single sign-on via Adobe Identity

To access AEM Author:

1. Navigate to `https://author-idfc-stage-65.adobecqms.net`
2. Authenticate with your Adobe ID via the IMS login screen

3. You will be directed to the AEM Start screen based on your assigned permissions

Edge Delivery Admin API Authentication

The Edge Delivery Admin API at <https://admin.hlx.page/> requires authentication for write operations (preview, publish, delete, config updates). Authentication is handled via:

- **API Key** – Provided by Adobe for programmatic access
- **User Token** – Obtained through the Sidekick browser extension or the Admin API authentication flow

For programmatic access, include the authentication header:

```
Authorization: token <your-api-key>
```

Sidekick Extension Setup

The AEM Sidekick browser extension provides a convenient UI for preview and publish operations. For this xwalk project, the Sidekick is configured to open the Universal Editor for content editing.

To install and configure the Sidekick:

1. Install the AEM Sidekick extension from the Chrome Web Store
2. Add the project configuration:
 - **Organization:** aemsites
 - **Site:** idfc
3. The Sidekick configuration includes:
 - **editUrlLabel:** "AEM Editor" – Labels the edit button for the Universal Editor
 - **editUrlPattern** with `?cmd=open` – Opens the Universal Editor when the edit button is clicked
4. The Sidekick will display Preview, Publish, and Edit buttons when browsing the site

4. User Management

User management in this xwalk project spans two tiers. Both must be configured correctly for a user to be able to author and publish content.

Tier 1: AEM Author Users

AEM Author users are managed through the AEM instance administration at <https://author-idfc-stage-65.adobecqms.net>. These users can access the Universal Editor and manage content within the JCR repository.

User Roles:

Role	Description	AEM Group
Content Author	Creates and edits pages using the Universal Editor	content-authors
Content Editor	Reviews and approves content changes	content-editors
DAM User	Manages assets in the DAM	dam-users
Administrator	Full AEM instance access	administrators

Adding an AEM Author User:

1. Navigate to AEM Author > Tools > Security > Users
2. Click "Create User" and provide Adobe ID email
3. Assign the user to appropriate groups (content-authors, dam-users, etc.)

4. Set content path permissions under /content/idfc-edge/ and /content/dam/idfc-edge/

Removing an AEM Author User:

1. Navigate to AEM Author > Tools > Security > Users
2. Select the user account
3. Remove from all groups or deactivate the account

Tier 2: Edge Delivery Config Service Users

Edge Delivery Config Service users can perform preview and publish operations through the Sidekick and Admin API. These users are managed via the Config Service API.

Adding an Admin User to Config Service:

```
curl -X POST "https://admin.hlx.page/config/aemsites/sites/idfc/admin.json" \
-H "Authorization: token <admin-api-key>" \
-H "Content-Type: application/json" \
-d '{
  "admin": {
    "users": ["user@example.com"]
  }
}'
```

Listing Current Admin Users:

```
curl -X GET "https://admin.hlx.page/config/aemsites/sites/idfc.json" \
-H "Authorization: token <admin-api-key>"
```

Adding an Author/Preview User:

Authors who need Sidekick preview/publish access must be added to the Config Service:

```
curl -X POST "https://admin.hlx.page/config/aemsites/sites/idfc/access.json" \
-H "Authorization: token <admin-api-key>" \
-H "Content-Type: application/json" \
-d '{
  "access": {
    "users": ["author@example.com"]
  }
}'
```

Removing a User from Config Service:

Update the user list to exclude the user, then POST the updated list:

```
curl -X POST "https://admin.hlx.page/config/aemsites/sites/idfc/access.json" \
-H "Authorization: token <admin-api-key>" \
-H "Content-Type: application/json" \
-d '{
  "access": {
    "users": ["remaining-user@example.com"]
  }
}'
```

Complete User Onboarding Checklist

For a new author to be fully operational, complete these steps:

1. Add user to Adobe Admin Console with appropriate product profile
2. Add user to AEM Author instance groups (content-authors, dam-users)
3. Set JCR permissions for /content/idfc-edge/ and /content/dam/idfc-edge/
4. Add user to Edge Delivery Config Service access list
5. Have the user install and configure the AEM Sidekick browser extension
6. Verify the user can open the Universal Editor and perform preview/publish

5. Content Operations

All content operations in this xwalk project follow the pattern: edit in AEM Author via the Universal Editor, then use the Admin API (or Sidekick) to push content to the Edge Delivery CDN.

Preview a Page

Previews content from AEM Author to the preview tier (aem.page):

```
curl -X POST "https://admin.hlx.page/preview/aemsites/idfc/main/{path}" \
-H "Authorization: token <api-key>"
```

Example – Preview the homepage:

```
curl -X POST "https://admin.hlx.page/preview/aemsites/idfc/main/" \
-H "Authorization: token <api-key>"
```

Example – Preview a credit card page:

```
curl -X POST "https://admin.hlx.page/preview/aemsites/idfc/main/credit-card/metal-credit-card/mayura" \
-H "Authorization: token <api-key>"
```

Publish a Page (Go Live)

Publishes content from the preview tier to the live tier (aem.live):

```
curl -X POST "https://admin.hlx.page/live/aemsites/idfc/main/{path}" \
-H "Authorization: token <api-key>"
```

Check Page Status

Returns the current status of a page across preview and live tiers:

```
curl -X GET "https://admin.hlx.page/status/aemsites/idfc/main/{path}" \
-H "Authorization: token <api-key>"
```

The response includes timestamps for when the page was last previewed and published, along with the source content hash.

Unpublish a Page

Remove a page from the preview tier:

```
curl -X DELETE "https://admin.hlx.page/preview/aemsites/idfc/main/{path}" \
-H "Authorization: token <api-key>"
```

Remove a page from the live tier:

```
curl -X DELETE "https://admin.hlx.page/live/aemsites/idfc/main/{path}" \
-H "Authorization: token <api-key>"
```

Bulk Operations

Preview or publish multiple pages in a single request:

Bulk Preview:

```
curl -X POST "https://admin.hlx.page/preview/aemsites/idfc/main/*" \
-H "Authorization: token <api-key>" \
-H "Content-Type: application/json" \
-d '{
  "paths": [
    "/",
    "/credit-card/metal-credit-card/mayura",
    "/dept-best-practice/form/contact-form"
  ]
}'
```

Bulk Publish:

```
curl -X POST "https://admin.hlx.page/live/aemsites/idfc/main/*" \
-H "Authorization: token <api-key>" \
-H "Content-Type: application/json" \
-d '{
  "paths": [
    "/",
    "/credit-card/metal-credit-card/mayura",
    "/dept-best-practice/form/contact-form"
  ]
}'
```

Content Operations Summary Table

Operation	Method	Endpoint
Preview	POST	/preview/aemsites/idfc/main/{path}
Publish	POST	/live/aemsites/idfc/main/{path}
Unpublish (Preview)	DELETE	/preview/aemsites/idfc/main/{path}
Unpublish (Live)	DELETE	/live/aemsites/idfc/main/{path}
Status	GET	/status/aemsites/idfc/main/{path}
Bulk Preview	POST	/preview/aemsites/idfc/main/*
Bulk Publish	POST	/live/aemsites/idfc/main/*

All endpoints are relative to the base URL `https://admin.hlx.page/`.

6. Path Mappings

The `paths.json` file defines how JCR content paths in AEM Author map to URL paths on the Edge Delivery CDN. This is a critical configuration for administrators to understand, as it controls which content is accessible and at what URLs.

Current Path Mappings

The project defines 14 path mappings:

JCR Path	Edge Delivery URL	Purpose
<code>/content/idfc-edge/</code>	<code>/</code>	Content root – all pages under this tree are delivered
<code>/content/idfc-edge/config</code>	<code>/config.json</code>	Site configuration sheet
<code>/content/idfc-edge/headers</code>	<code>/.helix/headers.json</code>	HTTP response headers configuration
<code>/content/idfc-edge/configuration</code>	<code>/.helix/config.json</code>	Helix configuration
<code>/content/idfc-edge/redirects</code>	<code>/redirects.json</code>	URL redirect rules
<code>/content/idfc-edge/metadata</code>	<code>/metadata.json</code>	Global metadata definitions
<code>/content/idfc-edge/designs/design</code>	<code>/design.json</code>	Global design tokens/configuration
<code>/content/idfc-edge/credit-card/metal-credit-card/mayura/design</code>	<code>/credit-card/metal-credit-card/mayura/design.json</code>	Mayura product-specific design tokens
<code>/content/idfc-edge/experiment-metadata</code>	<code>/experiment-metadata.json</code>	A/B testing and experimentation metadata
<code>/content/idfc-edge/api/test</code>	<code>/api/test.json</code>	API test endpoint
<code>/content/idfc-edge/dept-best-practice/form/contact-form</code>	<code>/dept-best-practice/form/contact-form.json</code>	Contact form definition
<code>/content/dam/idfc-edge/icons/</code>	<code>/icons/</code>	Icon assets from DAM
<code>/content/dam/idfc-edge/videos/</code>	<code>/videos/</code>	Video assets from DAM

Content Includes

The `includes` array specifies which JCR paths are included in the Edge Delivery content scope:

Include Path	Purpose
<code>/content/idfc-edge/</code>	All page content
<code>/content/dam/idfc-edge/icons/</code>	Icon assets
<code>/content/dam/idfc-edge/videos/</code>	Video assets

Key Points for Administrators

- **Content Root:** All pages must reside under `/content/idfc-edge/` in AEM Author to be deliverable.
- **DAM Assets:** Only icons and videos from `/content/dam/idfc-edge/` are included. Other DAM paths are not served by Edge Delivery.
- **Config Sheets:** Configuration pages (config, headers, redirects, metadata, design) are mapped as JSON endpoints. Changes to these pages in AEM Author must be previewed and published like any other content.
- **Product-Specific Designs:** The Mayura credit card has its own `design.json` mapping, allowing product-specific design tokens independent of the global design.

- **Adding New Mappings:** To add a new path mapping, update `paths.json` in the GitHub repository and trigger a code sync.

7. Cache Management

CDN Cache Architecture

Edge Delivery Services uses a multi-tier caching architecture. Content served on `aem.live` is cached at the CDN edge. When content is republished, the CDN cache is automatically invalidated for the affected paths. However, there are scenarios where manual cache management is required.

Purging Cache via Admin API

Purge a specific path:

```
curl -X DELETE "https://admin.hlx.page/cache/aemsites/idfc/main/{path}" \
-H "Authorization: token <api-key>"
```

Purge the entire site cache:

```
curl -X DELETE "https://admin.hlx.page/cache/aemsites/idfc/main/" \
-H "Authorization: token <api-key>"
```

When to Purge Cache

Scenario	Action
After updating headers configuration	Purge affected paths or full site
After updating redirects	Purge the <code>redirects.json</code> path
After emergency content fixes	Republish the page (auto-purges), then verify
After <code>config.json</code> changes	Purge full site if config affects all pages
After <code>design.json</code> updates	Purge affected section of the site
Stale content visible after publish	Purge the specific path and republish

Cache Invalidation Best Practices

1. **Prefer republish over manual purge** – Publishing a page automatically invalidates its CDN cache. Manual purging should be reserved for edge cases.
2. **Purge selectively** – Avoid full-site cache purges in production unless absolutely necessary, as they temporarily increase origin load.
3. **Verify after purge** – Always verify the live site after a cache purge to ensure content is rendering correctly.
4. **Configuration changes** – Changes to `.helix/headers.json`, `metadata.json`, or `config.json` may require a broader purge since these affect multiple pages.

8. Code Operations

Code Deployment Pipeline

Code changes in the IDFC project follow this pipeline:

1. Developer pushes code to the `aemsites/idfc` GitHub repository (main branch)
2. GitHub Actions runs the lint workflow (ESLint for JavaScript/JSON, Stylelint for CSS)
3. Edge Delivery detects the code change and syncs automatically (or manually via Admin API)
4. Updated code is available on preview (`aem.page`) and live (`aem.live`) tiers

Triggering Code Sync via Admin API

If code changes are not reflected automatically, trigger a manual code sync:

```
curl -X POST "https://admin.hlx.page/code/aemsites/idfc/main" \
-H "Authorization: token <api-key>"
```

Checking Code Status

```
curl -X GET "https://admin.hlx.page/status/aemsites/idfc/main" \
-H "Authorization: token <api-key>"
```

Build Scripts

The project uses several build scripts defined in `package.json`:

Script	Command	Purpose
<code>lint:js</code>	<code>eslint . --ext .json,.js,.mjs</code>	Lint JavaScript and JSON files
<code>lint:css</code>	<code>stylelint blocks/**/*.css styles/*.css</code>	Lint CSS files
<code>lint</code>	<code>npm run lint:js && npm run lint:css</code>	Run all linters
<code>build:json</code>	<code>npm-run-all -p build:json:models build:json:definitions build:json:filters</code>	Build component JSON files
<code>build:json:models</code>	<code>merge-json-cli</code>	Merge component model fragments into <code>component-models.json</code>
<code>build:json:definitions</code>	<code>merge-json-cli</code>	Merge component definition fragments into <code>component-definition.json</code>
<code>build:json:filters</code>	<code>merge-json-cli</code>	Merge component filter fragments into <code>component-filters.json</code>

GitHub Actions

The project uses a lint-on-push workflow. When code is pushed to any branch, the GitHub Actions workflow runs ESLint and Stylelint to validate code quality before it reaches Edge Delivery.

Code Rollback

To rollback a code change:

1. Identify the last known good commit in the GitHub repository
2. Create a revert commit or reset the main branch to the good commit
3. Push to GitHub
4. Trigger a code sync if needed: `POST /code/aemsites/idfc/main`
5. Verify on preview, then publish affected pages

9. Config Service Management

The Edge Delivery Config Service stores the site-level configuration that controls how Edge Delivery operates for this project. In xwalk projects such as this one, the Config Service is the authoritative source for the content source configuration – there is no repository-level file that points to the content source. The AEM Author markup endpoint is configured entirely through the Config Service.

Reading Current Configuration

```
curl -X GET "https://admin.hlx.page/config/aemsites/sites/idfc.json" \
-H "Authorization: token <api-key>"
```

This returns the complete site configuration including content source, CDN settings, headers, access controls, and more.

Key Configuration Sections

Content Source

The content source configuration confirms this is a markup (xwalk) project. In xwalk projects, the content source (AEM Author markup endpoint) is configured through the Edge Delivery Config Service, not through a file in the repository. This is a fundamental architectural difference from document-based projects.

```
{
  "contentSource": {
    "url": "https://author-idfc-stage-65.adobecqms.net/bin/franklin.delivery/aemsites/
idfc/main",
    "type": "markup",
    "suffix": ".html"
  }
}
```

This tells Edge Delivery to pull content from the AEM Author Franklin delivery servlet. **Do not change the type to anything other than markup** unless migrating the entire content architecture.

Headers Configuration

HTTP response headers are managed in two places:

1. **Config Service** – Global headers applied to all responses
2. **AEM Author** – The headers page at `/content/idfc-edge/headers`, mapped to `/.helix/headers.json`

To update headers via Config Service:

```
curl -X POST "https://admin.hlx.page/config/aemsites/sites/idfc/headers.json" \
-H "Authorization: token <api-key>" \
-H "Content-Type: application/json" \
-d '{
  "headers": {
    "/*": {
      "X-Frame-Options": "SAMEORIGIN",
      "X-Content-Type-Options": "nosniff"
    }
  }
}'
```

CDN Configuration

CDN-specific settings including caching behavior, edge compute rules, and origin configuration are managed through the Config Service.

Access Control

The Config Service manages two access lists:

- **admin** – Users who can manage configuration and perform all operations
- **access** – Users who can preview and publish content via Sidekick

Updating Site Configuration

To update any section of the site configuration:

```
curl -X POST "https://admin.hlx.page/config/aemsites/sites/idfc.json" \
-H "Authorization: token <api-key>" \
-H "Content-Type: application/json" \
-d '{
  "<section>": {
    "<key>": "<value>"
  }
}'
```

Always read the current configuration before making changes, and test on preview before publishing.

10. AEM Author Administration

Content Tree Management

All page content resides under `/content/idfc-edge/` in the AEM JCR repository. The content tree structure directly maps to the URL structure on Edge Delivery (after path mapping).

Key content tree locations:

AEM Path	Purpose
<code>/content/idfc-edge/</code>	Site root (homepage)
<code>/content/idfc-edge/config</code>	Site configuration sheet
<code>/content/idfc-edge/headers</code>	HTTP headers configuration
<code>/content/idfc-edge/configuration</code>	Helix configuration

AEM Path	Purpose
/content/idfc-edge/redirects	URL redirects
/content/idfc-edge/metadata	Global metadata
/content/idfc-edge/designs/design	Global design tokens
/content/idfc-edge/credit-card/	Credit card product pages
/content/idfc-edge/dept-best-practice/	Departmental best practices

DAM Asset Management

Digital assets are managed under /content/dam/idfc-edge/ in the AEM DAM. Only specific subdirectories are included in Edge Delivery:

- /content/dam/idfc-edge/icons/ – Mapped to /icons/ on Edge Delivery
- /content/dam/idfc-edge/videos/ – Included for video delivery

When uploading new icons or videos:

1. Upload to the appropriate folder in DAM via AEM Author
2. Preview the asset via Admin API or Sidekick
3. Publish the asset to make it available on the live tier
4. Reference the asset in page content using the Edge Delivery path (e.g., /icons/my-icon.svg)

Component Model Management

Component models define the authoring experience in the Universal Editor. These are code-managed files deployed via GitHub – they are NOT managed in AEM Author.

Component Definition Files:

File	Purpose
component-definition.json	Defines available components and their groups (Default Content, Sections, Blocks)
component-models.json	Defines the authoring fields and properties for each component
component-filters.json	Defines which child components are allowed inside container components

These files are built from fragment files in the /models/ directory using the build:json npm scripts:

```
npm run build:json
```

This merges fragments from: - models/_component-models.json into component-models.json - models/_component-definition.json into component-definition.json - models/_component-filters.json into component-filters.json

Available Block Components (26 blocks):

The project includes these custom blocks: Accordion, Anchor Nav, Banner, Cards, Cards Icon, Cards Testimonial, Category Nav, CC Hero Slider, Columns, CTA Sticky, Embed, Fragment, Hero, Hero Heritage CC, Hotspot, Library Metadata, Link To UPI, Overview, Section Title, Steps, Table, Tabs, and Tabs UPI Link.

Each block has an associated resourceType of core/franklin/components/block/v1/block and may define child item components for multi-item blocks.

Updating Component Models:

1. Edit the appropriate fragment file in the `models/` directory
2. Run `npm run build:json` to regenerate the merged JSON files
3. Commit and push to GitHub
4. Trigger a code sync if needed
5. Authors will see the updated component definitions in the Universal Editor

Template Management

Templates define the page structure available to authors in the Universal Editor. They are managed under the `/templates/` directory in the GitHub repository and control what sections and blocks are pre-populated when an author creates a new page.

11. Indexing and Sitemap

Query Index (helix-query.yaml)

The project maintains a query index for content discovery. The configuration is:

```
version: 1

indices:
  pages:
    include:
      - '**'
    exclude:
      - '**.json'
    target: /query-index.json
    properties:
      lastModified:
        select: none
        value: parseTimestamp(headers["last-modified"], "ddd, DD MMM YYYY hh:mm:ss GMT")
      robots:
        select: head > meta[name="robots"]
        value: attribute(el, "content")
```

Key details:

- **Index name:** `pages`
- **Output:** `/query-index.json`
- **Scope:** All pages (`**`) excluding JSON files (`**.json`)
- **Properties indexed:** `lastModified` (from HTTP headers) and `robots` (from meta tags)

The query index is automatically rebuilt when pages are published. To force a rebuild, republish pages or trigger via the Admin API.

Sitemap (helix-sitemap.yaml)

The sitemap is generated from the query index:

```
sitemaps:
  default:
    source: /query-index.json
```

```
destination: /sitemap.xml
lastmod: YYYY-MM-DD
```

- **Source:** The pages query index at /query-index.json
- **Output:** /sitemap.xml
- **Last Modified Format:** ISO date format (YYYY-MM-DD)
- **Production Sitemap:** Referenced in robots.txt at <https://www.idfcfirst.bank.in/sitemap.xml>

Note: The current robots.txt disallows all crawlers (Disallow: /), which is typical for a staging environment. Before going to production, this should be updated to allow appropriate crawlers.

Reindexing

To trigger a reindex of the site content:

1. Republish the pages that need reindexing
2. The query index at /query-index.json will be updated automatically
3. The sitemap at /sitemap.xml will regenerate from the updated index

12. Monitoring and Troubleshooting

Common Admin Tasks Quick Reference

Task	API Call
Preview a page	POST /preview/aemsites/idfc/main/{path}
Publish a page	POST /live/aemsites/idfc/main/{path}
Check page status	GET /status/aemsites/idfc/main/{path}
Unpublish from preview	DELETE /preview/aemsites/idfc/main/{path}
Unpublish from live	DELETE /live/aemsites/idfc/main/{path}
Purge CDN cache	DELETE /cache/aemsites/idfc/main/{path}
Trigger code sync	POST /code/aemsites/idfc/main
Read site config	GET /config/aemsites/sites/idfc.json
Bulk preview	POST /preview/aemsites/idfc/main/* with paths array
Bulk publish	POST /live/aemsites/idfc/main/* with paths array

All endpoints are relative to <https://admin.hlx.page/>.

Emergency Procedures

Full Site Republish

If the site is in an inconsistent state, perform a full republish:

1. **Identify affected paths** – Use the status API to check which pages are out of date
2. **Bulk preview all content:**


```
curl -X POST "https://admin.hlx.page/preview/aemsites/idfc/main/*" \
  -H "Authorization: token <api-key>" \
  -H "Content-Type: application/json" \
  -d '{"paths": ["/**"]}'
```

3. **Verify preview** – Check <https://main-idfc-aemsites.aem.page/> for correctness

4. **Bulk publish all content:**

```
curl -X POST "https://admin.hlx.page/live/aemsites/idfc/main/*" \
  -H "Authorization: token <api-key>" \
  -H "Content-Type: application/json" \
  -d '{"paths": ["/**"]}'
```

5. **Purge CDN cache** if stale content persists:

```
curl -X DELETE "https://admin.hlx.page/cache/aemsites/idfc/main/" \
  -H "Authorization: token <api-key>"
```

Code Rollback

If a code deployment causes issues:

1. Identify the last working commit hash in the GitHub repository
2. Create a revert commit: `git revert <bad-commit-hash>`
3. Push the revert to the main branch
4. Trigger code sync: `POST /code/aemsites/idfc/main`
5. Verify on preview, then republish affected pages

Content Sync Issues Between AEM Author and Edge Delivery

If content edited in AEM Author is not appearing correctly on Edge Delivery:

1. **Verify AEM Author content** – Open the page in the Universal Editor and confirm the content is saved correctly
2. **Check the Franklin delivery servlet** – Access <https://author-idfc-stage-65.adobecqms.net/bin/franklin.delivery/aemsites/idfc/main/{path}.html> to verify the markup is correct
3. **Re-preview the page** – Trigger a fresh preview via Admin API
4. **Check path mappings** – Ensure the content path is within `/content/idfc-edge/` and is correctly mapped in `paths.json`
5. **Check status** – Use `GET /status/aemsites/idfc/main/{path}` to see if the page has been previewed/published

Debugging Path Mapping Issues

Path mappings in `paths.json` are a common source of issues. To debug:

1. **Verify the JCR path exists** – Navigate to the path in AEM Author
2. **Check the mapping** – Ensure the path is under `/content/idfc-edge/` (or `/content/dam/idfc-edge/` for assets)
3. **Check includes** – The path must be covered by one of the `includes` entries in `paths.json`
4. **Test the mapped URL** – Access the corresponding Edge Delivery URL to see if content resolves
5. **Config sheet mappings** – If the content is a configuration sheet (config, headers, metadata, etc.), verify it maps to the correct `.json` endpoint

Common Issues and Resolutions

Issue	Likely Cause	Resolution
Page shows 404 on preview	Path not under /content/idfc-edge/ or not previewed	Verify path and trigger preview
Stale content after edit	Preview/publish not triggered after AEM Author edit	Re-preview and republish
Blocks not rendering	Component model mismatch	Run <code>npm run build:json</code> , push, code sync
Icons missing	Icon not in /content/dam/idfc-edge/icons/	Upload to correct DAM path and preview
CSS/JS not updating	Code sync not triggered	Trigger code sync via Admin API
Config changes not applied	Config sheet not republished	Preview and publish the config page
Links pointing to ww2 domain	Legacy links in content	Content uses <code>ww2.idfcfirst.bank.in</code> ; the link rewriting in <code>scripts.js</code> auto-converts to <code>www</code>
User cannot preview/publish	Missing from Config Service	Add user to Edge Delivery Config Service access list
User cannot edit content	Missing AEM Author access	Add user to AEM Author groups and set permissions

13. Security Configuration

ESLint Security Rules

The project enforces comprehensive security rules through ESLint plugins configured in `.eslintrc.js`:

Secure Coding Plugin (eslint-plugin-secure-coding)

These rules prevent common server-side and application-level vulnerabilities:

Rule	Severity	Purpose
<code>no-hardcoded-credentials</code>	error	Prevents credentials in source code
<code>no-redos-vulnerable-regex</code>	error	Prevents ReDoS-vulnerable regular expressions
<code>no-unsafe-deserialization</code>	error	Prevents unsafe deserialization
<code>no-improper-sanitization</code>	error	Enforces use of approved sanitizers (DOMPurify.sanitize, encodeURIComponent, etc.)
<code>no-format-string-injection</code>	error	Prevents format string injection
<code>no-unchecked-loop-condition</code>	error	Prevents infinite loops from unchecked conditions
<code>no-unlimited-resource-allocation</code>	error	Prevents DoS from unbounded resource allocation
<code>no-xpath-injection</code>	error	Prevents XPath injection
<code>no-graphql-injection</code>	error	Prevents GraphQL injection
<code>no-xxe-injection</code>	error	Prevents XML External Entity injection

Rule	Severity	Purpose
detect-non-literal-regexp	warn	Flags dynamic regular expressions
detect-object-injection	warn	Flags bracket notation property access
no-improper-type-validation	warn	Flags improper type checks
no-missing-authentication	warn	Flags unauthenticated endpoints
no-sensitive-data-exposure	warn	Flags potential data exposure
no-pii-in-logs	warn	Flags PII in log statements

Browser Security Plugin (eslint-plugin-browser-security)

These rules address client-side browser security concerns:

Rule	Severity	Purpose
no-innerhtml	error	Prevents innerHTML without DOMPurify sanitization
no-eval	error	Prevents eval() usage
require-postmessage-origin-check	error	Enforces origin validation on postMessage
no-postmessage-wildcard-origin	error	Prevents wildcard origin in postMessage
no-sensitive-localstorage	error	Prevents sensitive data in localStorage
no-jwt-in-storage	error	Prevents JWT tokens in browser storage
no-http-urls	error	Enforces HTTPS URLs
require-https-only	error	Enforces HTTPS-only connections
no-clickjacking	error	Prevents clickjacking vulnerabilities

The full list includes 30+ browser security rules covering cookies, WebSockets, service workers, CSP, and more.

DOMPurify Sanitization

The project uses DOMPurify 3.3.1 for HTML sanitization throughout the codebase. This is critical for preventing DOM XSS attacks when rendering editor content.

Usage pattern:

```
import { DOMPURIFY } from './scripts.js';

// In scripts.js -- loads DOMPurify on demand
await ensureDOMPurify();
element.innerHTML = window.DOMPurify.sanitize(htmlContent, DOMPURIFY);
```

Key integration points:

- scripts/scripts.js – Loads DOMPurify and uses it when injecting page content
- scripts/aem.js – Uses DOMPurify when building block cell content from markup
- scripts/editor-support.js – Sanitizes content from the Universal Editor before DOM injection
- scripts/dompurify.min.js – The DOMPurify 3.3.1 library

The MAX_ITERATION_LIMIT constant (set to 500) is also used throughout the codebase to prevent DoS from excessive loop iterations (CWE-400).

Link Rewriting Rules

The project includes automatic link rewriting to handle legacy domain references:

- **ww2.idfcfirst.bank.in** links are automatically rewritten to **www.idfcfirst.bank.in**
- Relative links are resolved against the production origin <https://www.idfcfirst.bank.in>
- PDF links and `my.idfcfirst` links are automatically opened in new tabs

This is implemented in the `makeProdUrl()` function in `scripts/scripts.js`.

Content Security Policy

The `head.html` file includes a commented-out Content Security Policy meta tag with strict-dynamic and nonce-based script loading:

```
<!-- https://www.aem.live/docs/csp-strict-dynamic-cached-nonce -->
<meta http-equiv="Content-Security-Policy"
      content="script-src 'nonce-aem' 'strict-dynamic' 'unsafe-inline' http: https;;
              base-uri 'self'; object-src 'none';" move-to>
```

This CSP is currently disabled but can be activated when the site moves to production. It uses the AEM Edge Delivery cached nonce approach for strict-dynamic script loading.

14. Resources

Adobe Documentation

Resource	URL
AEM Edge Delivery Services Overview	https://www.aem.live/docs/
Universal Editor Documentation	https://experienceleague.adobe.com/docs/experience-manager-cloud-service/content/implementing/developing/universal-editor/introduction.html
Admin API Reference	https://www.aem.live/docs/admin.html
Config Service API	https://www.aem.live/docs/config-service-setup
Sidekick Extension	https://www.aem.live/developer/tutorial#sidekick
Component Model Definitions	https://www.aem.live/developer/component-model-definitions
Path Mappings (paths.json)	https://www.aem.live/docs/paths
Indexing Configuration	https://www.aem.live/developer/indexing
Sitemap Configuration	https://www.aem.live/developer/sitemap
Crosswalk (xwalk) Projects	https://www.aem.live/developer/markup-sections-blocks
CSP Strict Dynamic Nonce	https://www.aem.live/docs/csp-strict-dynamic-cached-nonce

Project-Specific References

Resource	Location
GitHub Repository	https://github.com/aemsites/idfc
AEM Author Instance	https://author-idfc-stage-65.adobecqms.net

Resource	Location
Edge Delivery Preview	https://main-idfc-aemsites.aem.page/
Edge Delivery Live	https://main-idfc-aemsites.aem.live/
Config Service	https://admin.hlx.page/config/aemsites/sites/idfc.json
Query Index	https://main-idfc-aemsites.aem.live/query-index.json
Sitemap	https://main-idfc-aemsites.aem.live/sitemap.xml

This guide was prepared for the IDFC FIRST Bank Edge Delivery Services (xwalk) project administered under the aemsites organization. All API calls reference the `main` branch. For branch-specific operations, replace `main` with the target branch name in all Admin API endpoints.